STUDENT PRACTICAL EXAM STUDY GUIDE

Software Testing & Quality Assurance (STQA)

Practical No 8 : Object Identification & Counting Using Selenium WebDriver

1. Use your own GitHub account. Use the repository provided by your teacher.
2. Select the green Code button, open the Codespaces tab, and select Create codespace on main.
3. Wait for the Codespace and VS Code web editor to finish loading.
4. Check required tools using below commands one by one –

java -version
mvn -version
chromium --version
chromedriver --version

5. If maven, chromium, chrome driver are not present, then run the below commands one by one –

sudo apt-get update
sudo apt-get install -y maven chromium chromium-driver

6. Now, verify point 4 again
7. Start the server

python3 -m http.server 8000

OR IF THE SERVER DOES NOT START, run below command to start java server
jwebserver -p 8000 -d /workspaces/STQA-Exam- P8-ObjectCounting

8. Complete the three student TASKS in the java file.

// TASK 1: Count all images
int imageCount = driver.findElements(By.tagName("img")).size();

// TASK 2: Count all buttons
int buttonCount = driver.findElements(By.tagName("button")).size();

// TASK 3: Count all input fields
int inputCount = driver.findElements(By.tagName("input")).size();

Save the java file.

9. Open a new terminal and Run the command –

mvn test

Check that the reported values are consistent with the elements present on the webpage. A message saying BUILD SUCCESS with the correct object count should be displayed in the console.

Practical No 9 : List and ComboBox item verification using Selenium WebDriver

1. Use your own GitHub account. Use the repository provided by your teacher.
2. Select the green Code button, open the Codespaces tab, and select Create codespace on main.
3. Wait for the Codespace and VS Code web editor to finish loading.
4. Check required tools using below commands one by one –

RUN all environment checks –

===== JAVA =====

java -version

===== MAVEN =====

mvn -version

===== GOOGLE CHROME =====

google-chrome --version

===== CHROMEDRIVER =====

chromedriver --version

=====check PRACTICAL FILES =====

ls -la

expected output – devcontainer, src, pom.xml and readme should be listed

find src -type f

expected output – it should list the path of the java file and the html file

Start the Web Server
python3 -m http.server 8000

If it does not start, start java web server

jwebserver -p 8000 -d /workspaces/STQA-Exam-P9-List-ComboBox/src/test/resources

5. Open the java file and complete the student tasks -

Task 1 –

WebElement comboBox =

driver.findElement(By.id(“comboBox”));

Task 2 –

Select select = new Select(comboBox);

select.selectByVisibleText(“Selenium”);

6. In the second terminal, run the test –

mvn test

A message saying **BUILD SUCCESS** with the output dislaying **‘Combobox found, Option selected: Selenium’** should be displayed in the console.

Practical No 10 : Load Testing Using Apache JMeter

1. Open **Command Prompt** and verify Java:

   java -version

   You should see - openjdk version "21.x.x"

2. Then type : **javac -version**

   You should see - javac 21.x.x

3. Verify whether JMeter folder is available in C drive
4. Open the JMeter folder.
5. Open the folder : bin
6. Double-click : jmeter.bat

7. A black Command Prompt window may appear first. **Do not close it.**

8. After a few seconds, the **Apache JMeter GUI** should open.

9. Create a folder for the practical - C:\STQA-JMeter

10. In the above folder, Create a webpage using HTML and save the file as index.html. While saving the file, set **Save as type** to: all files

11. Start the server from the command prompt. First type : **cd C:\STQA-JMeter**

12. Then run : **java -m jdk.httpserver**

    You should see - Binding to loopback address 127.0.0.1, port 8000

13. Test the web page

Open Chrome/Edge and enter - **http://localhost:8000**

14. Verify that the **STQA Load Testing Application** page is displayed.

15. **Keep the Command Prompt open** while performing the JMeter test.

Perform all the steps as given in the practical document from **point # 6**

Different load conditions should be created using Thread Group, and response time, throughput, and error percentage should be observed using JMeter reports and graphs.

In the practical exam paper, Students should write 3 scenarios with different number of users, ramp up period and loop count and analyze the response time and throughput. Explanation of load testing, response time and throughput is required

Practical No 11 : Bug Tracking and Defect Lifecycle Management using JIRA

********** Students should have a atlassian.com account **************

Students are expected to create 3 defects in JIRA with proper title, description, Expected result, actual result, severity and priority. Students may change the status of the defect and understand the defect life cycle. Generate the 'defect summary' in the tool. Write all the defects along with the summary in the exam paper. Also, defect lifecycle diagram is to be drawn.

Practical No 12 : Checkbox Identification and Validation

1. Use your own GitHub account. Use the repository provided by your teacher.
7. Select the green Code button, open the Codespaces tab, and select Create codespace on main.
8. Wait for the Codespace and VS Code web editor to finish loading.
9. Check required tools using below commands one by one –

RUN all environment checks –

===== JAVA =====

java -version

===== MAVEN =====

mvn -version

===== GOOGLE CHROME =====

google-chrome --version

===== CHROMEDRIVER =====

chromedriver --version

Start the Web Server

jwebserver -p 8000 -d /workspaces/STQA-Exam-P12-CheckboxValidation/src/test/resources

Open the java file and complete the student tasks -

Task 1 –

```
 for (WebElement checkbox : checkboxes) {
   if (checkbox.isSelected()) {
      checkedCount++;
   } else
{
    uncheckedCount++;
   }
}
```

Task 2 -

```
for (WebElement checkbox : checkboxes) {
    if (checkbox.isSelected()) {
      checkedCount++;
    } else {
       uncheckedCount++;
  }
}
```

In the second terminal, run the test –

mvn test

A message saying **BUILD SUCCESS** with the output displaying **the number of checkbox (checked, unchecked, total)** should be displayed in the console.

Practical No 13 : Dynamic Web Table Handling and Sorting Validation

1. Use your own GitHub account. Use the repository provided by your teacher.
2. Select the green Code button, open the Codespaces tab, and select Create codespace on main.
3. Wait for the Codespace and VS Code web editor to finish loading.
4. Check required tools using below commands one by one
5. **RUN all environment checks –**
   ===== JAVA =====
   java -version
   ===== MAVEN =====
   mvn -version
   ===== GOOGLE CHROME =====
   google-chrome --version
   ===== CHROMEDRIVER =====
   chromedriver --version

Start the Web Server

jwebserver -p 8000 -d /workspaces/STQA-Exam-P13-DynamicWebTable/src/test/resources

Open the java file and complete the student tasks -

Task 1 –

```
rows = driver.findElements(
        By.cssSelector("#tableBody tr")
);
for (WebElement row : rows) {
List<WebElement> cells = row.findElements(
            By.tagName("td")
    );
            System.out.println(
            cells.get(0).getText() + " | " +
            cells.get(1).getText() + " | " +
            cells.get(2).getText() + " | " +
            cells.get(3).getText() + " | " +
            cells.get(4).getText()
```

```
    );
}
```

Task 2 –

```
Select resultFilter = new Select(
        driver.findElement(By.id("resultFilter"))
);
resultFilter.selectByVisibleText("Pass");
filteredRows = driver.findElements(
        By.cssSelector("#tableBody tr")
);
```

In the second terminal, run the test –

mvn test

A message saying **BUILD SUCCESS** with the output is displayed. Table data extraction, sorting validation, and filtered-result verification are successfully performed.

Practical No 14 : File Upload and Download Automation

1. Use your own GitHub account. Use the repository provided by your teacher.
2. Select the green Code button, open the Codespaces tab, and select Create codespace on main. Wait for the Codespace and VS Code web editor to finish loading. Check required tools using below commands one by one

3. Verify the Working Directory

Execute: **pwd**

Expected output : /workspaces/STQA-Practical-14-FileUploadDownload

If you are not in this directory, navigate to the repository folder before continuing.

4. Check Java Installation

Execute : **java -version**

Java 21 or later is recommended for this practical. The tested environment uses Microsoft OpenJDK 25.

Check the Java compiler : **javac -version**

Both commands should display the installed Java version.

If Java is missing

On Ubuntu, install Java 21:

sudo apt-get update
sudo apt-get install -y openjdk-21-jdk

Verify the installation:

java -version
javac -version

5. Check Maven Installation

Execute : **mvn -version**

The output should display the Maven version, Java version, and operating system.

If Maven is missing

Execute:

sudo apt-get update
sudo apt-get install -y maven

Verify : **mvn -version**

6. Check Google Chrome and ChromeDriver

Selenium requires a browser and a compatible driver. This practical uses Google Chrome for Testing and ChromeDriver.

Check whether Chrome is available in the system path : **which google-chrome**

Check Chromium : **which chromium**

which chromium-browser

Check ChromeDriver : **which chromedriver**

These commands may return no output in Codespaces. This does not necessarily mean that Chrome is unavailable.

7. Check the Ubuntu Version

Execute:

cat /etc/os-release | head

The tested environment uses Ubuntu 24.04 LTS.

8. Install Required Chrome Dependencies

Chrome requires certain Linux libraries to run in Codespaces.

Check whether the libraries are available:

ldconfig -p | grep -E 'libatk-1.0.so.0|libatk-bridge-2.0.so.0|libcups.so.2|libxkbcommon.so.0|libasound.so.2|libgbm.so.1|libXcomposite.so.1|libXdamage.so.1|libXfixes.so.3|libXrandr.so.2|libatspi.so.0'

If these dependencies are missing, install them using:

sudo apt-get update && sudo apt-get install -y libatk1.0-0 libatk-bridge2.0-0 libcups2 libxkbcommon0 libasound2t64 libgbm1 libxcomposite1 libxdamage1 libxfixes3 libxrandr2 libatspi2.0-0

Verify again:

ldconfig -p | grep -E 'libatk-1.0.so.0|libatk-bridge-2.0.so.0|libcups.so.2|libxkbcommon.so.0|libasound.so.2|libgbm.so.1|libXcomposite.so.1|libXdamage.so.1|libXfixes.so.3|libXrandr.so.2|libatspi.so.0'

The output should list the available libraries.

9. Verify Python 3

Python is used to start the local web server required by this practical.

Execute : **python3 --version**

If Python 3 is missing, install it:

sudo apt-get update
sudo apt-get install -y python3

Verify : **python3 --version**

10. Check the Practical Files

Execute : **ls**

Verify the Maven project file : **ls pom.xml**

Check the Java test file:

ls src/test/java/practical/FileUploadDownloadTest.java

Check the HTML page:

ls src/test/resources/file-upload-download.html

Check the sample upload file:

ls src/test/resources/upload-files/sample-upload.txt

Check the sample download file:

ls src/test/resources/download-files/sample-download.txt

All five files should be present.

11. Open the java file and complete the student tasks -

TASK 1 (FILE UPLOAD)

```
File uploadFile = Paths.get(
        "src", "test", "resources",
        "upload-files", "sample-upload.txt"
).toAbsolutePath().toFile();
assertTrue("Sample upload file must exist.",
        uploadFile.exists());
driver.findElement(By.id("fileInput"))
        .sendKeys(uploadFile.getAbsolutePath());
driver.findElement(By.id("uploadButton")).click();
```

TASK 2 (FILE DOWNLOAD)

```
driver.findElement(By.id("downloadLink")).click();
```

TASK 3 (ROBOT CLASS)

```
Robot robot = new Robot();
    robot.setAutoDelay(200);
    robot.keyPress(KeyEvent.VK_SHIFT);
    robot.keyPress(KeyEvent.VK_A);
    robot.keyRelease(KeyEvent.VK_A);
    robot.keyRelease(KeyEvent.VK_SHIFT);
```

12. Start the Local Web Server

Important: Start the web server before running mvn test. The test opens the webpage at http://127.0.0.1:8000/file-upload-download.html.

In the first terminal, execute:

python3 -m http.server 8000 --directory src/test/resources

Expected message:

Serving HTTP on 0.0.0.0 port 8000 ...

Keep this terminal open and running.

If Python reports that port 8000 is already in use

Check whether a server is already running:

ps -ef | grep '[h]ttp.server 8000'

If the existing server is serving this practical's src/test/resources directory, keep using it. Do not start another server on the same port.

13. Verify the Web Server

Open a **second terminal** in the same Codespace. Keep the first terminal running the server.

Execute : **curl -I http://127.0.0.1:8000/file-upload-download.html**

Expected result : HTTP/1.0 200 OK

Verify the sample download file : **curl -I**
http://127.0.0.1:8000/download-files/sample-download.txt

Expected result : HTTP/1.0 200 OK

View the webpage in a browser

1. Open the **Ports** tab in Codespaces.
2. Forward port 8000 if it has not been forwarded already.
3. Open the forwarded port's URL.

The practical webpage should display the file upload form and download link.

14. Run the Selenium Tests

In the second terminal, make sure you are in the project directory : **pwd**

Then execute : **mvn test**

On the first run, Maven may download dependencies, while Selenium Manager may download Chrome and ChromeDriver. Wait until the command finishes.

Students should receive BUILD SUCCESS. Students should also verify file upload and download is working correctly.

Practical No 15 : TestNG Framework Implementation and Reporting

1. Use your own GitHub account. Use the repository provided by your teacher.
2. Select the green Code button, open the Codespaces tab, and select Create codespace on main. Wait for the Codespace and VS Code web editor to finish loading. Check required tools using below commands one by one

3. Check that you are inside the practical folder -

pwd
ls

4. **Check Java installation**

Execute : **java -version**

Expected: Java version information, preferably Java 21 or later.

5. **Check Maven installation**

Execute : **mvn -version**

6. **Check Chrome and ChromeDriver**

Run these commands one at a time:

which google-chrome
which chromium
which chromium-browser
which chromedriver

If these are not available, it is OK.

7. **Install Chrome's required system dependencies**

Chrome may fail to start in a new Codespace if required Linux libraries are missing.

Execute the following command:

sudo apt-get update && sudo apt-get install -y libatk1.0-0 libatk-bridge2.0-0 libcups2 libxkbcommon0 libasound2t64 libgbm1 libxcomposite1 libxdamage1 libxfixes3 libxrandr2 libatspi2.0-0

Wait until the installation finishes and the terminal prompt returns.

If the packages are already installed, that is fine.

8. **Open TestNGLoginTest.java file and complete student tasks**

STUDENT Task 1: Enter valid username and password

```
driver.findElement(By.id("username")).sendKeys("student");

driver.findElement(By.id("password")).sendKeys("test123");

driver.findElement(By.id("loginButton")).click();
```

STUDENT Task 2: Enter invalid username and password

```
driver.findElement(By.id("username")).sendKeys("wronguser");

driver.findElement(By.id("password")).sendKeys("wrongpass");

driver.findElement(By.id("loginButton")).click();
```

9. **Start the login webpage server**

In **Terminal 1**, execute:

cd /workspaces/STQA-Exam-P15-TestNG-Framework && python3 -m http.server 8000 --directory src/test/resources

Leave this terminal running. Do not press Ctrl+C.

The login page is available at port 8000.

Now, run the test in **Terminal 2:**

cd /workspaces/STQA-Exam-P15-TestNG-Framework

mvn clean test

10. **Start the TestNG report server**

Open a **third terminal**.

Execute:

cd /workspaces/STQA-Exam-P15-TestNG-Framework && python3 -m http.server 8001 --directory target/surefire-reports

Leave this terminal running too.

The report server is available at port 8001. The report files will be generated when the tests run in the next step.

11. **Open the login page**

1. Open the **Ports** tab in Codespaces.
2. If port 8000 is not listed, forward port 8000.
3. Click the globe/open-in-browser icon for port 8000.

The login page should display the title **STQA Student Login**.

Test credentials:

Field	Value
Username	student
Password	test123

The page should display a successful login message for the correct credentials and an error message for incorrect credentials.

The automated tests use a local HTML file, so the test execution does not depend on an external login website.

12. **Open the TestNG report**

1. Return to the **Ports** tab.
2. If port 8001 is not listed, forward port 8001.
3. Click the globe/open-in-browser icon for port 8001.
4. Add /index.html to the end of the port's browser URL if necessary.

The report URL should have this general format:

https://<your-codespace-port-8001-url>/index.html

The report should display the test results. If it was opened before the tests finished, refresh the page after Maven completes.

13. **Verify the XML test results**

Return to the terminal and execute:

grep -E 'verifyPageTitle|verifyUsernameField|verifyPasswordField|verifySuccessfulLogin|verifyInvalidLogin' target/surefire-reports/testng-results.xml

To check the result counts, execute:

grep -E '<suite|<test |<test-method|total=|passed=|failed=|skipped=' target/surefire-reports/testng-results.xml

Confirm that all five test methods have status="PASS" and that there are no failures or errors.